Assignment No-9

Q.1) Selection Sort

Input - [18, 22, 20, 19, 21]

Dry Run.

{18, 22, 20, 19, 21}

18	22	20	19	21
0	1	2	3	4

{18, 22, 20, 19, 21}

{18, 19, 20, 22, 21}

~~18~~ | 19

{18, 19, 20, 22, 21}

{18, 19, 20, 21, 22}

Code.

```
int n = arr.length
for(int i=0; i<arr n-1; i++){
        int minIndex = i;
    for(int j= i+1; j<n; j++){
        if (arr[j] < arr[i] {
            minIndex = j;
        }
    }

    int temp = arr[i]
    arr[i] = arr[minIndex]
arr[minIndex] = temp
}
```

Dry Run

i=0; minIndex = 0; j = 1

if (arr[j] < arr[i]){
  19 < 18

}

i = 0; ~~i++~~; ~~i++~~; j++; j = 2 minIndex = 0;

if (arr[j] < arr[i]){
  20 < 18

}

i=0; j++ j=3, minIndex=0;

if(arr[j] < arr[i]){

19 < 18

}

i=0; j++; j=4 minIndex=0;

if (arr[j] < arr[i])

21 < 18

}

j++ j=5 for loop end j

i++

i=1; j= i+1 j=2 minIndex=1;

if(arr[j] < arr[i]){

20 < 22

minIndex = j;

2

}

int temp= arr[i] → 22

~~int arr[minIndex]~~ =

arr[i] = arr[minIndex] → 20

arr[minIndex] = temp → 22

}

18	20	22	19	21

i=1; j++; j=3 minIndex=1;

if (arr[j] < arr[i]){

19 22

18	22	20	19	21
0	1	2	3	4

```
int n= arr.length;
  for (int i=0; i< n-1; i++){
      int minIndex= i;
      for(int j= i+1; j<n; j++){
          if(arr[j] < arr[minIndex]){
              minIndex = j;
          }
      }

      int temp= arr[i];
      int arr[i] = arr[minIndex];
      arr[minIndex] = temp;
  }
```

n = 5 i=0; j = i+1 = 1; minIndex = i;

if(arr[j] < arr[i]) { (18 < 22)
  minIndex = j; (1) ✗
}

j++ 2 if(arr[j] < arr[i]){ (22 < 18) ✗
  minIndex = j
}

j++ j=3 if (arr[3] < arr[0]){ (19 < 18) ✗
}

j++ j=4 if (arr[4] < arr[0]){ (21 < 18) ✗
}

minIndex = 0;
int temp = arr[0]
int arr[0] = arr[0]
arr[0] = arr[0]

18	22	20	19	21
0	1	2	3	4

i++

i=1 j=i+1=2; minIndex=i; n=4.

if(arr[2] < arr[1]){
20 22

minIndex = j;
2
}

i=1 j++ j=3 minIndex=2 n=4

if(arr[3] < arr[1]){
19 < 22

minIndex=3;

i=1 j++ j=4 minIndex=2; n=4

if(arr[4] < arr[1]){
21 < 22

minIndex=j;

minIndex=4;

i=1 j=i+1=2 minIndex=i n=4

if(arr[2] < arr[1]){
20 22

minIndex=2;
}

j=3 minIndex=2

if(arr[3] < arr[2]){
19 20

minIndex=3
}

j=4 If (arr[4] < arr[3]){
21 19 X

j=5 X

[12, 34, 36, 23, 45]

[12, 34, 36, 23, 45]

[12, 23, 36, 45, 34]

[12, 23, 34, 45, 36]

[12, 23, 34, 36, 45]

0 1 2 3 4

n = array length

n = 5

i = 0; j = i + 1 minIndex = 0; n = 5

if (arr[1] < arr[0]) {
12 < 34

minIndex = j;
}

i = 0; j = 2; minIndex = 1

if (arr[2] < arr[1]) {
36 < 12

}

j++ if (arr[3] < arr[1]) {
45 < 12

j = 4 if (arr[4] < arr[1]) {
23 < 12

i = 0

minIndex = 1;

int temp = arr[0]

arr[0] = arr[1]

arr[1] = temp

12	34	56	45	23
0	1	2	3	4

$i = 1$ $j = i+1 = 2$ minIndex = 1 $n = 5$;

if(arr[2] < arr[1]){
56 < 34
}

j++ if(arr[3] < arr[1]){
45 < 34

j++ if(arr[4] < arr[i]){
23 < 34
minIndex = j;

int temp = arr[1]
arr[1] = arr[4]
arr[4] = temp

i++

12	23	56	45	34
0	1	2	3	4

$i = 2$ $j = 3$ minIndex = 2; $n = 5$

if(arr[3] < arr[2]){
45 < 56
minIndex = 3
}

if(arr[4] < arr[3]){
34 < 45
minIndex = 4;
}

int temp = arr[2]
arr[2] = arr[4]
arr[4] = temp;

12	23	34	45	56
0	1	2	3	4

i++

i = 3 j = 4 minIndex = 3

if (arr[4] < arr[3]) {

~~x~~ 56 45

}

j = 5 ~~x~~

int temp = arr[3]

arr[3] = arr[3]

arr[3] = arr[3]

~~ts~~ | 12 | 23 | 34 | 45 | 56 |

Q.2) Bubble Sort.

1)

{30, 25, 27, 35, 29}.

1) {25, 30, 27, 35, 29}

{25, 27, 30, 35, 29

{25, 27, 30, 35, 29}

{25, 27, 30, 29, 35}

2) {25, 27, 29, 30, 35}

~~fo~~

code →

```
for (int i=0; i<n-1; i++){
    boolean swapped = false;
    for (int j=0; j<n-i-1; j++){
        if (arr[j] > arr[j+1]) {
            int temp = arr[j]
            arr[j] = arr[j+1];
            arr[j+1] = temp;
            swapped = true
        }
    }
    if (!swapped){
        break;
    }
}
```

18	19	20	22	21
0	1	2	3	4

Int temp = arr[1]; 22
arr[1] = arr[3] 19
arr[3] = temp; 22

i = 2 j = i + 2 = 3 n = 5 minIndex = 2;

if (arr[3] < arr[2]){
22 20

i = 2 j = 4 minIndex = 2;

if (arr[4] < arr[2]){
21 20

int temp = arr[2]
arr[2] = arr[2]
arr[2 = arr[2]

18	19	20	22	21
0	1	2	3	4

i = 3 j = i + 1 = 4 n = 4 minIndex = 3;

if (arr[4] < arr[3]){
21 22

minIndex = 4;

int temp = arr[3] → 22
arr[3] = arr[4] → 21
arr[4] = temp

18	19	20	21	22

30	25	27	35	29
0	1	2	3	4

i=0; j=0; n=4 5-0-1

if(arr[j] > arr[j+1]){
30 25
int temp = arr[0]
arr[1] = arr[0]
arr[0] = temp.
}

25, 30, 27, 35, 29.

i=0; j=1
if(arr[1] > arr[2]){
25 27
int temp = arr[i]
arr[i] = arr[2]
arr[2] = ~~arr[~~ temp;
}
j = 2
if(arr[2] > arr[3]){
}
j=3
if(arr[3] > arr[4]){
35 29
int temp = arr[3];
arr[3] = arr[4];
arr[4] = temp;

25, 30, 27, 29, 35
0 1 2 3 4

i=1. j=0; n=5-1-1 n=3

if(arr[j] > arr[2]){
25 30
}

i=1

i=2.

i=1 j=1

n=5

if (arr[1] > arr[2])
30 27

int temp = arr[1] 27;
arr[1] = arr[2]
arr[2] = ~~arr~~ temp.
}

25, 27, 30, 29, 35.

j=2,

if (arr[2] > arr[3]){
30 > 29

int temp = arr[2]
arr[2] = arr[3]
arr[3] = temp;
}

25, 27, 29, 30, 35,

j=3, n=5-1-1 = 3

i=2. j=1, n=5-2-1 = n=2.

if (arr[1] > arr[2]) {
27 > 30

j=2 if (arr[2] > arr[3]) {
29 > 30
}

j=3 if (arr[3] > arr[4] {
30 > 35

if (! swapped) {
break
}

0) {10, 15, 25, 30, 40}

1) {10, 15, 25, 30, 40}

{10, 15, 25, 30, 40}

{10, 15, 25, 30, 40}

{10, 15, 25, 30, 40}

10	15	25	30	40

2) {10, 15, 25, 30, 40}.

{10, 15, 25, 30, 40}

{10, 15, 25, 30, 40}

3) {10, 15, 25, 30, 40}

{10, 15, 25, 30, 40}

{10, 15, 25, 30, 40}

```
for(int i=0; i< n-1; i++){
    boolean swapped= false;
    for(int j=0; j< n-i-1; j++)
        if(arr[j]> arr[j+1]){
            int temp= arr[j]
            arr[j]= arr[j+1]
            arr[j+1] = temp;
            swapped=true;
        }
    }
    if(!swapped){
        break
    }
```

i=0 j=0 swapped = false.

if (arr[0] > arr[1]){

10 > 15

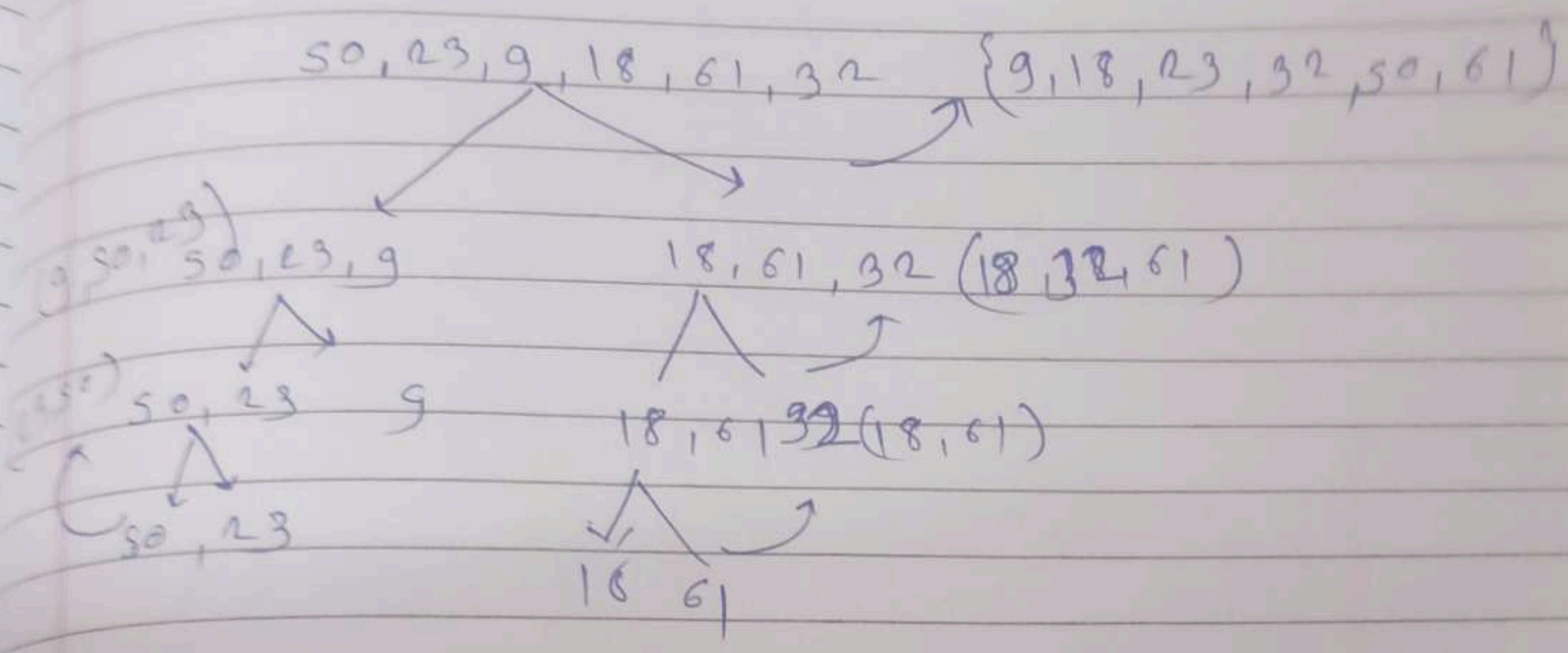
j++

j=1 if (arr[1] > arr[2]){

15 > 25

merge sort.

[50, 23, 9, 18, 61, 32]



Code

```
- public static void f(int[] arr, int low, int high){
      if (low>=high){return;}
      int mid = (low+high)/2

      f (arr, low, mid);
      f (arr, mid+1, high);
      merge(arr, low, mid, high);

public static void merge(int[]arr, int low, int mid
                         int high){

      int [] merged = new int [high-low+1];
      int blue = low
      int green = mid+1;
      int red = 0;
```

Q.2) {12, 23, 34, 45, 56}

i=01; current element = 23, left = 0;

while (0>=0 && 12>23){

}

arr[left+1] = current

12, 23, 34, 45, 56

i=2 currentele = 34 left = 1

while (1>=0 && 23>34){

}

arr[left+1] = current

{12, 23, 34, 45, 56}

i=3 current = 45, left = 2

while (2>=0 && 34>45){

}

arr[left+1] = current.

{12, 23, 34, 45, 56}

i=4 ~~wh~~ current = 56 left = 3

while (3>=0 && 45>56){

}

arr[left+1] = current

{12, 23, 34, 45, 56}

i=3 current ele = 19 ; left = 1

while(1 >= 0 && arr[left] > currentele){
arr[left+1] = arr[left];

| 18 | 20 | 20 | 22 | 21 |

left --;

while(0 >= 0 && arr[left] > current){

arr[left+1] = currentele;

| 18 | 19 | 20 | 22 | 21 |

i=4; current ele = 21; left = 3

while(3 >= 0 && arr[left] > 21){
arr[left+1] = arr[left]

| 18 | 19 | 20 | 22 | 22 |

left --;

i=4; currennt ele = 21 left = 2
while(2 >= 0 && 20 > 21)

arr[left+1] = current element;
arr[3] = 21

| 18 | 19 | 20 | 21 | 22 |

i=5
condition false

output = | 18 | 19 | 20 | 21 | 22 |

18	22	20	19	21
0	1	2	3	4

i = 1; currentelement = 22, left = 0;

~~curr~~

while (0 >= 0 && 18 >= 22){

arr[left+1] = arr[left];

left --;

}

18	~~18~~	~~20~~	19	21

arr[left+1] = current Element;

18	22	20	19	21

i = 2; currentElement = 20; left = 2-1 = 1

while (1 >= 0 && arr[1] >= currentelement){

22 20

arr[left+1] = arrleft;

18	22	22	19	21

left --;

}

0

arr[left+1] = current element;

while (0 >= 0 && arr[0] >= 20){

18 > 20

}

~~let~~ arr[1] = current element

18	20	22	19	21

i = 3; current element = 19; left = 3-1 = 2

while (2 >= 0 && arr[left] >= currentele){

20 19

arr[3] = arr[~~arr~~ 2];

left --;

18	20	22	22	21

}

j++ if(arr[2] > arr[3])
j=2. X 25 > 30

j++;
j=3 if(arr[3] > arr[4]){
X 30 > 40
}

→ if(!swapped){
break.
array is already sworted.

3) Insertion Sort.

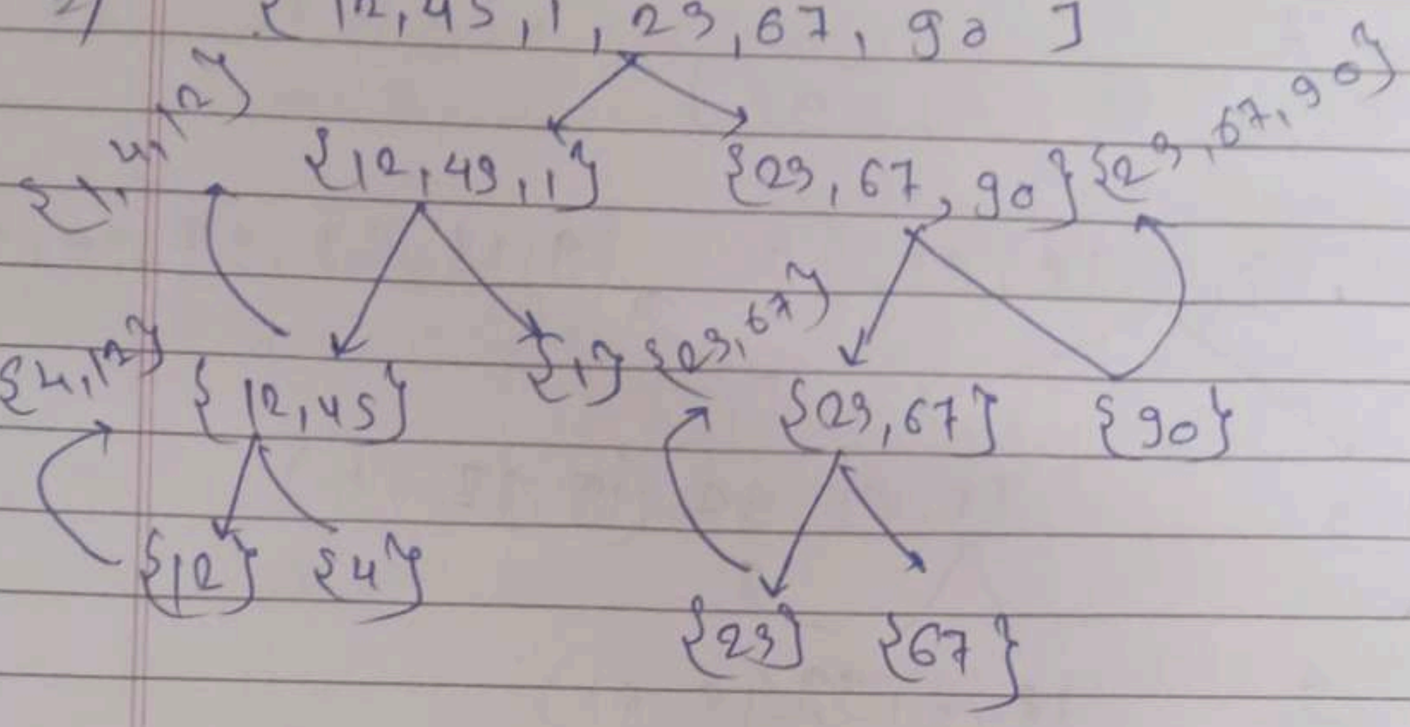
1)
{18, 22, 20, 19, 21}.

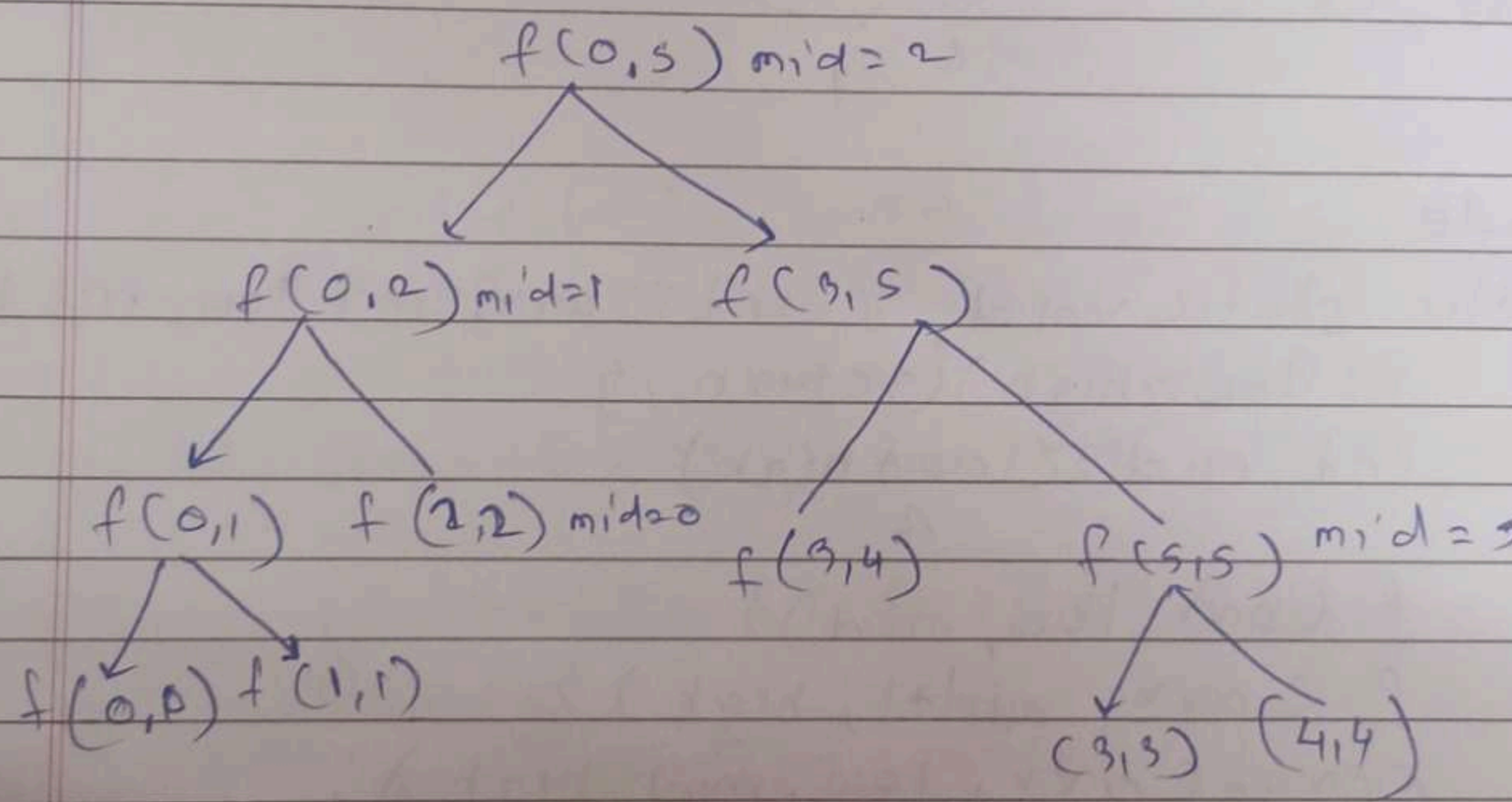
{18, 22, 20, 19, 21}
{18, 22, 20, 19, 21} key = 20
{18, 20, 22, 19, 21} key = 19
{18, 20, 22, 19, 21}
{18, 20, 22, 22, 21}
{18, 20, 20, 22, 21} key = 22
{18, 19, 20, 22, 21}
{18, 19, 20, 21, 4}
{18, 19, 20, 21, 22}

```
for( int i=1; i<n; i++){
    int current element = arr[i];
    int left = i-1;
    while( left>=0 && arr[left] > currentElement)
        arr[left+1] = arr[left]
        left--;
    }
    arr[left+1] = currentElement;
}
```

Merge Sort.

2) { 12, 45, 1, 23, 67, 90 }





merge(0,0,1)

↓

merge(0,1,2)

merge(3,3,4)

merge(3,4,5)

merge(0,2,5)

~~{75,~~

{55, 65, 75, 85, 95}

Q.5 Quick Sort.

1) {75, 85, 55, 95, 65}

55	65	75	85	95

55	65

85	95

{55} {65} {85} {95}

```
Public static void f (int[] arr, int low, int high) {
        if (low == high){
            return;
        }
        int pivot = pivotfinder(arr, low, high);
        f (arr, low, pivot-1)
        f (arr, pivot+1, high);
    }
~~void~~ int pivotfinder (int [] arr, int low, int high){
        int pivot = arr[low];
        int i = low;
        int j = high;

        while (i<j) {
            while (i<high && arr[i] ≤ pivot){
                i++;
            }
            while (j≥low && arr[j] > pivot){
                j--;
            }
            if (i<j){
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
    } return j;
```

{67, 12, 56, 42, 34}

i = 1 j = 2

while (1 < 2){ 12 42

while (1 ≤ 4 && arr[i] ≤ pivot){

i++;

}

i = 2 j = 2 56

while (2 > 0 && arr[i] > 42){

~~j++~~; j--

}

i = 2 j = 1

if (2 < 1){

~~int temp = arr[i] 56~~

~~arr[i] = arr[j] 42~~

~~arr[j] = temp 56~~

~~{67, 12, 42, 56, 34}~~

{67, 12, 56, 42, 34}

i = 2, j = 1

while(2 < 3){ 42

while (2 ≤ 4 && arr[i] ≤ 42){

i++;

}

i = 3 j = 0 56

while (3 ≥ 0 && arr[i] > 42){

j--

2] {42, 56, 12, 67, 34}
f(0,4)

int pivot = pivot finder(0,4)

int pivotfinder(low, high, arr){
int pivot = arr[low] = 42.
int i = low;
int j = high;

while(0<4){
while(0 && 42 ≤ pivot){
i++
}
i=1
while(4≥0 && arr[i] > pivot){
j--;
}
i=1 j=3 1<3
if(i<j){
int temp = arr[i];
~~int~~ arr[i] = arr[j] → 67;
arr[j] = temp;
i=1 j=3
{67, 56, 12, 42, 34}.
while(1<3){
while(1 && arr[i] ≤ 42){
i↑
while(3≥0 && arr[i] > pivot){
j--
}
i=1 j=2 1<2
if(i<j){
int temp = arr[i] → 56
arr[i] = arr[j] → 12
arr[j] = temp →

i = 2 j = 2 2<2

if(i<j){

~~while(~~ } pivot = 2.

while(2<2){

}

pivot = j j = 2

{55, 65, 75, 85, 95}

f(0,1) pivot = 0

↳ int pivot = pivotfinder(0,1);

int pivot = arr[low]; 55

int i = low = 0

int j = high = 1

while(0<1){

while(0≤1 && arr[i] ≤ 55){

i++;

}

while(1 ≥ 0) && arr[j] > pivot){ 65 55

~~i = 1 j = 1~~ j--

}

i = 1 j = 0.

if(1<j){

}

return j = 0; pivot = 0;

{55, 65, 75, 85, 95}

↳ ~~f(0, -1)~~

f(1, 4)

{75, 85, 55, 95, 65}

f(arr, 0, 4)

int pivot = pivot finder(0,4);

pivotfinder (0,4)

int pivot = arr[0], → 75

int i = low; 0;

int j = high = 4;

i=1 85 ≤ 75

while (0<4){ 75 75

while (0≤4 && arr[0] ≤ pivot){

i++; i=1

}

85 75

while (4≥0 && arr[i] > pivot){

~~j--~~ j--

} 3 int temp= 75.

if(1<3) arr[i] = arr[j] 95

1<3 arr[j] = temp=75

j=3 i=1

{95, 85, 55, 75, 65}

while(1<3){ ~~85~~ 95 75

while(1≤4 && arr[1] ≤ pivot){

~~85~~ 95

while (3≥0 && arr[1] > 75){

j-- j=2 int temp= 95

arr[1] = 55

arr[j] = 95

j=2 i=1

{55, 85, 95, 75, 65}

while (1<2){

~~85~~ 55

while (1≤4 && arr[1] ≤ 75){

i++;

85

while (2≥0 && arr[i] > 75){

j--

~~j=1~~

}